TabTank — Updated Build Plan

Turn your new tab into an aquarium. Draw your own fish and watch them swim.

Complete Build Plan — Chrome Extension

Table of Contents

- What You Already Have
- Phase 1 — Skeleton (Get Something On Screen)
- Phase 2 — Drawing Tool (painter.js)
- Phase 3 — Fish Spawning & Swimming (fish.js)
- Phase 4 — Ocean Backgrounds & Mode System
- Phase 5 — Bait Feature
- Phase 6 — Persistence (storage.js)
- Phase 7 — Bubbles (bubbles.js) — Optional Polish
- Phase 8 — Custom UI & Button Art
- Layer Stack & Z-Index Order
- Milestone Checklist

What You Already Have

manifest.json (Complete)

Manifest V3 Chrome extension that overrides the new tab page with `newtab.html`. Permissions for `storage` and `unlimitedStorage` are already declared. Icons referenced at 16, 48, and 128px sizes.

```
{
  "manifest_version": 3,
  "name": "TabTank",
  "version": "1.0.0",
  "description": "Turn your new tab into an aquarium. Draw your own fish and wat
  "chrome_url_overrides": {
    "newtab": "newtab.html"
  },
  "permissions": ["storage", "unlimitedStorage"],
  "icons": {
    "16": "icons/icon16.png",
    "48": "icons/icon48.png",
    "128": "icons/icon128.png"
  }
}
```

Git Repository Initialized

newtab.html (Skeleton Complete)

Basic structure with ocean div, fish canvas, draw panel (canvas + controls row), bait bucket, and script tags. Uses `<input type="color">` for color picking instead of individual color buttons.

css/style.css (Basic Layout Complete)

Body, ocean, fish canvas, draw panel, draw canvas, controls row, and bait bucket are all positioned and styled with basic CSS.

What's Still Missing

No JavaScript (painter.js, fish.js, storage.js, bubbles.js are empty). No background art assets. No mode system. No custom button art.

Phase 1 — Skeleton (Get Something On Screen)

Status: MOSTLY COMPLETE — HTML and basic CSS are done. Minor tweaks may be needed.

Step 1.1: File Structure

```
TabTank/
├── manifest.json              ✅ exists
├── newtab.html                ✅ exists
├── css/
│   └── style.css              ✅ exists
├── js/
│   ├── painter.js
│   ├── fish.js
│   ├── bubbles.js
│   └── storage.js
├── assets/
│   ├── backgrounds/
│   │   ├── ocean/             (standard ocean art)
│   │   └── pixel/             (pixel art backgrounds, 320×180)
│   └── ui/                    (custom button art)
└── icons/
    ├── icon16.png             ✅ exists (placeholder)
    ├── icon48.png             ✅ exists (placeholder)
    └── icon128.png            ✅ exists (placeholder)
```

Step 1.2: newtab.html

Complete. Contains: - `<div id="ocean">` — background container - `<canvas id="fishCanvas">` — fullscreen transparent fish swimming surface - `<div id="drawPanel">` with: - `<canvas id="drawCanvas" width="300" height="200">` — the fish drawing pad - `<div id="controls">` wrapping: - `<input type="color">` — full color picker - `<input type="range">` — brush size slider (1–20) - Eraser, Clear, Spawn buttons - `<div id="baitBucket">` - Script tags: storage.js, fish.js, painter.js, bubbles.js (in that order)

Step 1.3: style.css

Complete. Core layout rules:

SELECTOR	KEY PROPERTIES
`body`	`margin: 0; overflow: hidden; width: 100vw; height: 100vh; background: #0a1628;`
`#ocean`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 0;`
`#fishCanvas`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 1; pointer-events: none;`
`#drawPanel`	`position: fixed; bottom: 20px; right: 20px; z-index: 10; background: rgba(0,0,0,0.6); border-radius: 12px; padding: 12px; display: flex; flex-direction: column; align-items: center;`
`#controls`	`display: flex; align-items: center;`
`#drawCanvas`	`background: white; border-radius: 8px; cursor: crosshair;`
`#baitBucket`	`position: fixed; z-index: 5; cursor: pointer;`

> **Why `pointer-events: none` on fishCanvas?** So mouse clicks pass through the transparent fish canvas to elements below it (like the bait bucket). You'll toggle this back to `auto` temporarily when bait mode is active.

Step 1.4: Test It

1. Go to `chrome://extensions` in your browser
2. Enable **Developer mode** (toggle in the top right)
3. Click **Load unpacked** → select your TabTank folder
4. Open a new tab

You should see: a dark blue screen, a white drawing canvas in the bottom-right corner with buttons, and a bait bucket element. Nothing works yet — that's fine. You just proved the extension loads.

> **Testing Workflow (use this for every change going forward):** After every code change: go to `chrome://extensions`, click the refresh icon on your extension card, then open a new tab to see the changes.

Phase 2 — Drawing Tool (painter.js)

Step 2.1: Get the Canvas Context

- Grab `#drawCanvas` with `document.getElementById`
- Call `canvas.getContext('2d')` — store the result as `ctx`
- Set initial drawing values:
- `ctx.lineCap = 'round'` — rounds the ends of lines
- `ctx.lineJoin = 'round'` — rounds the corners where lines meet
- `ctx.lineWidth = 4` — default brush thickness
- `ctx.strokeStyle = '#000000'` — default color: black

Step 2.2: Implement Freehand Drawing

You need one variable: `let isDrawing = false`

Add four event listeners on the drawing canvas:

EVENT	WHAT IT DOES
`mousedown`	Set `isDrawing = true`. Call `ctx.beginPath()`, then `ctx.moveTo(e.offsetX, e.offsetY)`.
`mousemove`	If `isDrawing` is false, return early (do nothing). Otherwise: call `ctx.lineTo(e.offsetX, e.offsetY)`, then `ctx.stroke()`.
`mouseup`	Set `isDrawing = false`.

EVENT	WHAT IT DOES
`mouseleave`	Also set `isDrawing = false` (stops drawing if cursor leaves the canvas).

> **Test checkpoint:** Reload extension, open new tab, draw on the white canvas. You should see black lines following your mouse.

Step 2.3: Color Picker

Add a `change` event listener on your `<input type="color">` that sets `ctx.strokeStyle` to the picker's current value. Store the active color in a variable so the eraser can restore it later.

```
const colorPicker = document.getElementById('colorPicker');
colorPicker.addEventListener('input', function(e) {
    ctx.strokeStyle = e.target.value;
});
```

> **Test checkpoint:** Draw in different colors. The color picker should change the line color.

Step 2.4: Brush Size

Add an `input` event listener on your range slider. On change, set `ctx.lineWidth` to the slider's current value. A range of 1 to 20 works well.

> **Test checkpoint:** Move the slider and draw. Lines should get thicker/thinner.

Step 2.5: Eraser

Two approaches — pick one:

APPROACH	HOW	TRADEOFF
Simple	Eraser button sets `ctx.strokeStyle = '#ffffff'` (draws in white). When user picks a color again, it	Only works because the draw canvas has a

APPROACH	HOW	TRADEOFF
	switches back.	white background.
Better	Eraser button sets `ctx.globalCompositeOperation = 'destination-out'`. This erases to transparent. When user picks a color, set it back to `'source-over'`.	Works with any background color. Better if you ever change the canvas background.

Step 2.6: Clear Button

Click listener that calls `ctx.clearRect(0, 0, canvas.width, canvas.height)`.

If you're using the simple eraser approach (drawing in white), also fill with white after clearing:

```
ctx.fillStyle = 'white';
ctx.fillRect(0, 0, canvas.width, canvas.height);
```

Test checkpoint: Draw something, erase parts, clear, change colors and sizes. The drawing tool should feel complete and responsive at this point.

Phase 3 — Fish Spawning & Swimming (fish.js)

Step 3.1: Set Up the Fish Canvas

- Grab `#fishCanvas` with `getElementById`
- Get its 2D context: `const fishCtx = fishCanvas.getContext('2d')`
- Set the canvas `width` and `height` **attributes** to `window.innerWidth` and `window.innerHeight`
- Add a `resize` event listener on `window` that updates these dimensions when the browser resizes

> **Important: Canvas Resolution vs CSS Size** CSS `width: 100%` stretches the canvas visually, but the canvas internal resolution is set by its `width` and `height` attributes. If you only set CSS size, your fish will be blurry. You must set both the CSS size AND the element attributes to match the window dimensions.

Step 3.2: Create the Fish Array

`const fishArray = []`

Each fish object in this array will have the following properties:

PROPERTY	TYPE	PURPOSE
`image`	Image	The drawn fish as an Image object
`x`	Number	Current horizontal position
`y`	Number	Current vertical position
`speedX`	Number	Horizontal velocity (positive = right, negative = left)
`wobbleOffset`	Number	Random offset so fish don't bob in sync
`width`	Number	Rendered width on screen
`height`	Number	Rendered height on screen
`targetX`	Number or null	Bait target X (null = normal swimming)
`targetY`	Number or null	Bait target Y (null = normal swimming)

Step 3.3: Wire Up the Spawn Button

Add a click listener on the Spawn button that does the following, in order:

1. **Export the drawing:** `const dataURL = drawCanvas.toDataURL('image/png')` — this converts whatever the user drew into a base64-encoded PNG string.
2. **Create an Image object:** `const img = new Image()` then `img.src = dataURL`

3. **Wait for the image to load:** Everything below goes inside `img.onload = () => { ... }`. You must wait because the image isn't usable until it loads, even from a data URL.
4. **Pick a random starting position:**
5. `x` : either `-100` (start offscreen left) or `canvas.width + 100` (start offscreen right)
6. `y` : random between `50` and `canvas.height - 150` (keep fish away from very top and bottom)
7. **Pick random movement values:**
8. `speedX` : random between `0.5` and `2.5`. If starting from the right side, make it negative.
9. `wobbleOffset` : random between `0` and `Math.PI * 2`
10. **Set dimensions:** Scale the drawing down to about 80–120px wide. Maintain aspect ratio based on the draw canvas dimensions (e.g. if draw canvas is 300×200, a 100px wide fish would be ~67px tall).
11. **Push the fish object** into `fishArray` with all the properties listed in Step 3.2. Set `targetX` and `targetY` to `null`.
12. **Clear the drawing canvas** so the user can draw another fish.

> **Test checkpoint:** Draw a fish, click Spawn. Nothing will appear yet (no animation loop), but check the browser console (F12 → Console tab) — there should be no errors. Optionally add a `console.log(fishArray)` to verify the fish object was created.

Step 3.4: Build the Animation Loop

Create a function called `animate` that will run ~60 times per second. This is the core of the entire project.

Every frame, the function does:

1. **Clear the entire fish canvas:** `fishCtx.clearRect(0, 0, fishCanvas.width, fishCanvas.height)`
2. **Loop through** `fishArray` and for each fish, update its position and draw it:

Position update (normal swimming): `javascript fish.x += fish.speedX fish.y += Math.sin(Date.now() * 0.002 + fish.wobbleOffset) * 0.5` The first line moves the fish horizontally at constant speed. The second line uses `Math.sin()` to create a smooth wave — the `wobbleOffset` makes each fish bob at a different phase, the `0.002` controls how fast the wave oscillates, and the `0.5` controls the amplitude (how far up/down it bobs).

Edge detection: `javascript if (fish.x > fishCanvas.width + fish.width) fish.speedX *= -1; if (fish.x < -fish.width) fish.speedX *= -1;` When the fish moves fully offscreen on either side, reverse its direction.

Draw the fish: (see Step 3.5 for the flip logic)

1. **Request the next frame:** `requestAnimationFrame(animate)` at the end of the function.

Kick off the loop by calling `animate()` once when the page loads.

> **What is `requestAnimationFrame`?** This is the browser's built-in "call this function on the next screen refresh" method. It's smoother than `setInterval`, runs at the display's refresh rate (usually 60fps), and automatically pauses when the tab isn't visible (saves CPU).

Step 3.5: The Flip Logic in Detail

This is the trickiest part of the animation. When a fish swims left, the image needs to be mirrored horizontally so it faces the direction it's traveling.

When going left (`fish.speedX < 0`):

```
fishCtx.save();
fishCtx.translate(fish.x + fish.width, fish.y);   // move origin to fish's right
fishCtx.scale(-1, 1);                              // flip horizontally
fishCtx.drawImage(fish.image, 0, 0, fish.width, fish.height);
fishCtx.restore();
```

When going right (`fish.speedX > 0`):

```
fishCtx.drawImage(fish.image, fish.x, fish.y, fish.width, fish.height);
```

`ctx.save()` and `ctx.restore()` ensure the transform (translate + scale) only affects this one fish and doesn't corrupt drawing for the next fish in the loop.

> **Major test checkpoint:** Draw a fish, click Spawn. You should see it swimming across the screen, bobbing gently up and down, and flipping when it hits the edge. Spawn multiple fish — they should all move independently with different speeds and wobble patterns. **If this works, the hard part is done.**

Phase 4 — Ocean Backgrounds & Mode System

This phase is expanded from the original plan. Instead of one static background, you're building a **mode system** that lets users switch between different visual themes — including a pixel art mode.

Step 4.1: Create Your Art

Open Procreate, Figma, or whatever design tool you prefer.

You need art for at least two modes:

Standard Ocean Mode

Resolution: 1920×1080 or larger, exported as PNG or JPEG.

For animated depth, split into 2–3 layers and export each separately:

LAYER FILE	CONTENT	ANIMATION
`bg-deep.png`	The far background — deep water, gradient, distant elements	None (stays still)
`bg-foreground.png`	Close-up elements — coral, sand floor, seaweed, rocks	Slow horizontal sway
`bg-light.png`	Light rays / caustics (export with transparency as PNG)	Opacity pulse

Save into `assets/backgrounds/ocean/`.

Pixel Art Mode

Resolution: **320×180** — draw at this tiny resolution intentionally. The browser will scale it up to fullscreen with hard pixel edges, creating the chunky pixel art look.

Same layer approach if you want, or a single image is fine:

LAYER FILE	CONTENT	ANIMATION
`pixel-bg.png`	Full pixel art ocean scene at 320×180	None or subtle sway

Save into `assets/backgrounds/pixel/`.

> **Tip:** Even 2 layers (background + foreground) with a slight parallax sway makes a big difference. The contrast between a static layer and a moving layer creates a convincing sense of depth with almost zero effort.

Step 4.2: Add a Mode Toggle

Add a mode toggle button to your HTML. This could go in the controls row, or as a separate floating element. For example:

```
<button id="modeToggle">Switch Mode</button>
```

Step 4.3: Add Image Layers to HTML

Inside `<div id="ocean">`, add `<img>` tags for each layer. Give them classes like `ocean-layer-deep`, `ocean-layer-fg`, `ocean-layer-light`.

You'll have image tags for **each mode's backgrounds** — the mode system will show/hide the right set.

Step 4.4: Style the Layers

Each image layer needs these CSS properties:

```
.ocean-layer {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;    /* fills the screen without distortion */
}
```

`object-fit: cover` is critical — it ensures the image fills the entire viewport regardless of screen size, cropping edges if needed rather than leaving gaps.

Step 4.5: Pixel Mode CSS

The key to pixel art mode is one CSS property: `image-rendering: pixelated`.

Create a CSS class for pixel mode that applies to backgrounds and canvases:

```
body.pixel-mode #ocean img {
    image-rendering: pixelated;
}

body.pixel-mode #fishCanvas {
    image-rendering: pixelated;
}

body.pixel-mode #drawCanvas {
    image-rendering: pixelated;
}
```

When pixel mode is active, everything renders with hard pixel edges instead of smooth scaling. Your 320×180 backgrounds will look intentionally blocky and retro when blown up to fullscreen.

Optional — Pixel draw canvas: In pixel mode, you could also shrink the draw canvas resolution (e.g. `width="60" height="40"`) so drawings come out pixelated. JavaScript would swap the canvas dimensions when the mode changes.

Step 4.6: Mode Switching Logic (JavaScript)

In one of your JS files (or a new `modes.js`), add the mode toggle logic:

```
The mode toggle button click does:
1. Toggle a class on <body> (e.g. "pixel-mode")
2. Show/hide the correct background images
3. Optionally adjust the draw canvas resolution
4. Save the current mode to Chrome storage so it persists
```

The CSS handles the visual differences via the body class — when `body` has class `pixel-mode`, the pixel CSS rules kick in. When it doesn't, the standard rules apply. JavaScript just flips the class.

Step 4.7: Animate with CSS Keyframes

Define keyframe animations in your CSS:

Sway animation (for the foreground layer):

```
@keyframes sway {
    0%   { transform: translateX(-5px); }
    100% { transform: translateX(5px); }
}

.ocean-layer-fg {
    animation: sway 6s ease-in-out infinite alternate;
}
```

Light pulse animation (for the light rays layer):

```
@keyframes lightPulse {
    0%   { opacity: 0.25; }
    100% { opacity: 0.5; }
}

.ocean-layer-light {
    animation: lightPulse 4s ease-in-out infinite alternate;
}
```

The deep background layer gets **no animation** — it stays completely still. This contrast between static and moving layers is what creates the illusion of depth.

> **Test checkpoint:** Reload, open new tab. Your art should fill the screen with subtle animation. Fish should swim on top of it on the transparent canvas layer. Toggle between modes — backgrounds should switch and pixel mode should render with hard pixel edges.

Phase 5 — Bait Feature

Step 5.1: Bait Mode Toggle

You need a variable: `let baitMode = false`

Add a click listener on `#baitBucket` that does:

1. Set `baitMode = true`
2. Change the cursor on `#fishCanvas` to `crosshair` (or a custom cursor image if you want)
3. Set `fishCanvas.style.pointerEvents = 'auto'` — this is critical. Earlier you set `pointer-events: none` so clicks pass through the canvas. Now you need the canvas to capture clicks so you can detect where the user clicks for bait placement.

Step 5.2: Ocean Canvas Click Handler

Add a `click` event listener on `#fishCanvas`. The handler does:

1. **Guard clause:** If `baitMode` is false, return immediately (do nothing).
2. **Get click coordinates:** `const mouseX = e.clientX`, `const mouseY = e.clientY`
3. **Find the closest fish:**
4. Set `let closestFish = null` and `let closestDist = Infinity`
5. Loop through every fish in `fishArray`. For each:
   - Calculate the fish's center: `centerX = fish.x + fish.width / 2`, `centerY = fish.y + fish.height / 2`
   - Calculate distance: `Math.sqrt((centerX - mouseX) ** 2 + (centerY - mouseY) ** 2)`
   - If this distance is less than `closestDist`, update `closestFish` and `closestDist`
6. If no fish exist (array is empty), just reset bait mode and return
7. **Set the target on the closest fish:** `closestFish.targetX = mouseX`, `closestFish.targetY = mouseY`
8. **Reset bait mode:**
9. `baitMode = false`
10. Restore cursor to default
11. Set `fishCanvas.style.pointerEvents = 'none'` again

Step 5.3: Update the Animation Loop

In your `animate` function, modify the fish update logic. For each fish, **before** the normal swimming code, add a check:

If `fish.targetX !== null` (fish is chasing bait):

1. **Calculate angle to target:** `const angle = Math.atan2(fish.targetY - fish.y, fish.targetX - fish.x)`
2. **Set rush speed:** `const rushSpeed = 6` (much faster than the normal 0.5–2.5)
3. **Move toward target:** `javascript fish.x += Math.cos(angle) * rushSpeed; fish.y += Math.sin(angle) * rushSpeed;`
4. **Flip the fish image:** Base the direction on the target, not `speedX`. If `fish.targetX < fish.x`, the fish should face left (use the mirroring flip from Step 3.5). Otherwise face right.
5. **Check if the fish arrived:** Calculate distance to target. If distance is less than `15` pixels:
6. Set `fish.targetX = null` and `fish.targetY = null`
7. Update `fish.speedX` to continue in the direction it was rushing. If it was heading left, set `speedX` to a small negative random value. If right, a positive one. This prevents the fish from awkwardly snapping in the opposite direction.

Else (no target): Normal swimming logic (the wobble + drift from Step 3.4).

> **What does `Math.atan2` do?** `Math.atan2(dy, dx)` gives you the angle (in radians) from one point to another. Then `Math.cos(angle)` and `Math.sin(angle)` give you the X and Y components of movement in that direction. This makes the fish swim in a straight line toward the click, regardless of where it is on screen.
>
> **Test checkpoint:** Spawn a few fish, click the bait bucket, click somewhere on the ocean. The nearest fish should dart quickly toward that spot, then resume normal swimming when it arrives.

Optional polish for the bait feature: - **One-click flow alternative:** Skip the bucket entirely — just make right-click anywhere drop bait (simpler UX) - **All nearby fish react:** Instead of just the closest, find all fish within a certain radius - **Bait disappears visually:** Draw a small shrinking circle at the click point that fades out - **Eating animation:** Make the fish briefly scale up ~10% when it reaches the bait, then shrink back to normal size

---

Phase 6 — Persistence (storage.js)

Step 6.1: Save Function

Create a function called `saveFish()` that:

1. **Maps `fishArray` to serializable objects.** You can't store Image objects directly in Chrome storage. Instead, map each fish to a plain object containing only the data you need: `javascript { dataURL: fish.image.src, // the base64 PNG string x: fish.x, y: fish.y, speedX: fish.speedX, wobbleOffset: fish.wobbleOffset, width: fish.width, height: fish.height }`
2. **Save to Chrome storage:** `chrome.storage.local.set({ fish: serializedArray })`

Call `saveFish()` at the end of your Spawn button handler (after pushing to the array).

Step 6.2: Load Function

Create a function called `loadFish()` that:

1. Calls `chrome.storage.local.get('fish', (result) => { ... })`
2. If `result.fish` exists and is an array, loops through each saved fish object:
3. Create `const img = new Image()`
4. Set `img.src = saved.dataURL`
5. Inside `img.onload = () => { ... }`, push a full fish object into `fishArray` using the saved properties and the loaded image

Call `loadFish()` when the page loads, **before** starting the animation loop.

> **Why `img.onload`?** Even though the image source is a base64 data URL (not a network request), the browser still needs a moment to decode it. The `onload` callback ensures you don't try to draw the image before it's ready. The animation loop will just show an empty canvas for a few frames until the images finish loading — that's perfectly fine.

Step 6.3: Save Mode Preference

In addition to saving fish, save the current mode (standard vs pixel) so it persists across tabs:

```
chrome.storage.local.set({ mode: currentMode })
```

Load it on startup and apply the correct body class before showing anything.

Step 6.4: Delete a Fish (Optional but Recommended)

If the user draws something ugly, they'll want to remove it. Options in order of complexity:

METHOD	IMPLEMENTATION	COMPLEXITY
"Clear all fish" button	Empty `fishArray`, call `chrome.storage.local.remove('fish')`	Easy — do this for v1
Right-click a fish	On right-click, find nearest fish (same distance logic as bait), remove from array, save	Medium
Drag to trash icon	Much more complex, skip for v1	Hard

At minimum, add a "Clear all fish" button somewhere accessible.

> **Test checkpoint:** Spawn fish, close the tab, open a new tab. Your fish should reappear and continue swimming from roughly where they were. Switch modes, close tab, reopen — mode should persist too. This confirms persistence is working.

Phase 7 — Bubbles (bubbles.js) — Optional Polish

Step 7.1: Bubble Array

Create an array of bubble objects, each with:

PROPERTY	VALUE
`x`	Random position across the screen width
`y`	Start at the bottom of the screen

PROPERTY	VALUE
`radius`	Random between 2–6px
`speedY`	Random between 0.3–1.0 (how fast it rises)
`drift`	Random slight horizontal wobble value

Spawn 20–30 bubbles on page load.

Step 7.2: Animate Bubbles

Three approaches — pick one:

APPROACH	HOW	PROS/CONS
Add to fish canvas loop	After drawing fish, draw translucent white circles: `fishCtx.arc()` with `rgba(255,255,255,0.3)` fill	Simple, no extra elements. Slightly more canvas drawing per frame.
Separate canvas	A third canvas layered between ocean and fish, dedicated to bubbles	Clean separation. More DOM elements.
CSS-animated divs	Create small round `<div>`s with `background: rgba(255,255,255,0.3)`, animate them upward with CSS `@keyframes`, randomize `animation-duration` and `left` position. When animation ends, reset to bottom.	Easiest. No canvas needed. Performs fine for 20–30 bubbles.

The CSS approach is recommended for simplicity.

Phase 8 — Custom UI & Button Art

This is a new phase for personalizing the look and feel of the extension beyond the defaults.

Step 8.1: Draw Your Buttons in Procreate

Design custom button art for each control: - Eraser button - Clear button - Spawn button - Bait bucket - Mode toggle

Export each as a PNG with transparency. Save into `assets/ui/`.

For pixel mode, draw a second set at low resolution (e.g. 32×16 per button) and save those separately.

Step 8.2: Apply Custom Button Art with CSS

Strip the default button appearance and use your art as background images:

```
button {
    border: none;
    background: none;
    cursor: pointer;
}

#spawnBtn {
    width: 60px;
    height: 30px;
    background-image: url('../assets/ui/spawn-btn.png');
    background-size: contain;
    background-repeat: no-repeat;
}
```

For pixel mode buttons, override with the pixel art versions:

```
body.pixel-mode #spawnBtn {
    background-image: url('../assets/ui/pixel/spawn-btn.png');
    image-rendering: pixelated;
}
```

Step 8.3: Hover & Active States

Add visual feedback so buttons feel responsive:

```
#spawnBtn:hover {
    transform: scale(1.1);       /* slightly bigger on hover */
}
```

```
#spawnBtn:active {
    transform: scale(0.95);      /* slightly smaller when clicked */
}
```

Or swap to a different image for the hover/pressed states if you draw those too.

> **Note:** For this phase to work, your buttons need `id` attributes in the HTML so CSS can target them individually. Add IDs like `id="eraserBtn"`, `id="clearBtn"`, `id="spawnBtn"` to each button.
>
> **Test checkpoint:** All buttons should display your custom art instead of browser defaults. Hover and click should feel responsive. Pixel mode should swap to pixel art buttons.

Layer Stack & Z-Index Order

This is how all the visual elements stack on top of each other:

```
┌──────────────────────────────────────────────┐
│ z-index: 0   Ocean background images (your art)│
│ z-index: 1   Fish canvas (transparent,fullscreen)│
│ z-index: 2   Bubbles (if using divs/extra canvas)│
│ z-index: 5   Bait bucket                      │
│ z-index: 10  Drawing panel                     │
└──────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────┐
│  bg-deep.png  (static)                        │
│    ↕ bg-foreground.png (slow sway ~5px)       │
│      ✨ bg-light.png (opacity pulse)          │
│                                               │
│        🐟 drawn fish swimming on canvas       │
│        🫧 procedural bubbles floating up      │
│                                               │
│  🪣 bait bucket        ┌──────────────┐       │
│                        │ 🎨 Draw your │       │
│                        │   fish here  │       │
│                        │   [Spawn]    │       │
```

Milestone Checklist

Work through these in order. Each one is independently testable — don't move to the next until the current one works.

#	MILESTONE	HOW TO VERIFY
1	Extension loads, blue screen + empty panel visible	New tab shows your layout with dark blue background and drawing panel
2	Can draw on the small canvas in multiple colors/sizes	Freehand drawing works with color changes, brush sizes, eraser, and clear
3	Spawn creates a swimming fish	Fish moves across the screen and flips horizontally at the edges
4	Multiple fish swim independently	Spawn 5+ fish, all have different speeds and wobble at different phases
5	Ocean art shows as background with CSS animation	Your hand-drawn art fills the screen with subtle sway and light pulse
6	Mode toggle switches between standard and pixel art	Backgrounds swap, pixel mode renders with hard pixel edges
7	Bait click makes nearest fish dart to cursor position	Fish rushes to click point then smoothly resumes normal swimming
8	Fish and mode persist across new tabs	Close tab, open new one, all previously spawned fish are still there in the same mode
9	Bubbles float upward	Visual polish — translucent circles rise from bottom of screen

#	MILESTONE	HOW TO VERIFY
10	Custom button art replaces browser defaults	All buttons display your Procreate art, with pixel variants in pixel mode

Quick Reference — Key APIs You'll Use

Canvas 2D Context

METHOD / PROPERTY	WHAT IT DOES
`getContext('2d')`	Gets the 2D drawing context from a canvas element
`beginPath()`	Starts a new drawing path
`moveTo(x, y)`	Moves the pen to a position without drawing
`lineTo(x, y)`	Draws a line from current position to (x, y)
`stroke()`	Renders the current path as a visible line
`clearRect(x, y, w, h)`	Erases a rectangular area
`drawImage(img, x, y, w, h)`	Draws an image onto the canvas
`save()` / `restore()`	Saves/restores the current transform state
`translate(x, y)`	Moves the canvas origin point
`scale(x, y)`	Scales drawing. `scale(-1, 1)` flips horizontally
`toDataURL('image/png')`	Exports canvas content as a base64 PNG string
`strokeStyle`	Sets the line/stroke color
`lineWidth`	Sets the line thickness in pixels

METHOD / PROPERTY	WHAT IT DOES
`lineCap`	Sets line end shape (`'round'` for smooth ends)
`globalCompositeOperation`	Controls how new drawings blend with existing content

Math Functions for Animation

FUNCTION	WHAT IT DOES
`Math.sin(value)`	Returns a value between -1 and 1 in a wave pattern. Used for the swimming wobble.
`Math.atan2(dy, dx)`	Returns the angle (in radians) from one point to another. Used for bait chasing direction.
`Math.cos(angle)`	X-component of movement in a given direction
`Math.sin(angle)`	Y-component of movement in a given direction
`Math.sqrt(x**2 + y**2)`	Distance between two points (Pythagorean theorem)
`Math.random()`	Random number between 0 and 1. Scale and offset as needed.

Chrome Extension APIs

API	WHAT IT DOES
`chrome.storage.local.set({ key: value })`	Saves data to the extension's local storage (persists across tabs and browser restarts)
`chrome.storage.local.get('key', callback)`	Retrieves saved data. The callback receives an object with your key.
`chrome.storage.local.remove('key')`	Deletes a specific key from storage

Browser Animation

API	WHAT IT DOES
`requestAnimationFrame(callback)`	Schedules a function to run on the next screen refresh (~60fps). Pauses when tab is hidden. Use this instead of `setInterval` for smooth animation.
`Date.now()`	Returns current timestamp in milliseconds. Use as input to `Math.sin()` for time-based animation that doesn't depend on frame rate.

CSS Pixel Art Mode

PROPERTY	WHAT IT DOES
`image-rendering: pixelated`	Scales images with hard pixel edges instead of smooth interpolation. Apply to backgrounds, canvases, and button images in pixel mode.